Time Travel Debugger-Phase 1(Server)

Progress:

4-October (this day I implement):

- User-defined Stack (linked list based: push, pop, peek, isEmpty, depth, snapshot_into) in server.cpp.
- Stage 1 / Pass 0x0 (Validation): readSourceLine, firstWord, secondWord, validateProgram. Uses my own Stack to track the open func. A func is pushed, a func_end pops it, a func found while the stack is not empty means a nested declaration and is rejected, and a leftover entry at the end means a missing func_end.
- Stage 2 / Pass 0x1 (Resolve): writeResolveRecord, readResolveRecord, resolveProgram. Every source line is stored in resolve.bin . FuncEntry remembers where each function starts, PendingPatch remembers each CALL's offset field, and after the whole file is written every CALL is patched with its target function's offset. Returns the offset of main.
- Errors handled: nested func, func without func_end, func_end without func, no main, call to an undefined function, duplicate function name.

From where I will start tomorrow:

- Stage 3: doubly linked Timeline, tokenizeLine, buildSnapshot, executeProgram
- Stage 4: writeHeader, writeTdbg

I didn’t take time to my project on October 5 and October 6 because I am busy preparing quizzes and also doing some assignments and labs on these days.

7-October (this day I implement):

- User-defined doubly linked list Timeline (record, begin, getStepCount, destructor that frees all snapshots)
- Stage 3 / Pass 0x2 (Execution): tokenizeLine, buildSnapshot, executeProgram. Supports set, add, sub, mul, div, call, func_end.
- Errors handled during execution: division by zero, undefined variable, stack overflow (depth above 64), argument count mismatch, unknown instruction.

- Stage 4 / Pass 0x3 (Serialization): writeHeader completed, writeTdbg writes a header placeholder, then every snapshot while saving its starting position in the index array, then the index array, then goes back and rewrites the header with stepCount and indexOffset.

Time Travel Debugger-Phase 2(Client and LB)

8-November (this day I implement):

- Implemented `helper.h` to establish foundational utility functions and shared data structures.
- Initiated the core logic setup for both the Server and Load Balancer modules.

9-November (this day I implement):

- Finalized the complete implementation of the Server, Load Balancer, and Client components.
- Debugged and resolved all compilation, linker, and runtime errors to achieve a fully working system.